Введение в верификацию и анализ программ

Модели программ

ITMO more than a UNIVERSITY

2026



Модели программ

- Модели программ:
  - Текстовые модели
  - Структурные модели
  - Поведенческие модели
- Структурные модели основаны на
  - Синтаксисе исходного языка
  - Структуре программы
- Поведенческие модели
  - Основаны на семантике конструкций языков программирования
- Гибридные модели
  - Объединяют структурные и поведенческие свойства



Модели программ

- Текстовые
  - Текст программы как модель
- Структурные модели
  - Дерево разбора (синтаксическое дерево, parsing tree, concrete syntax tree)
  - Абстрактное синтаксическое дерево (abstract syntax tree, AST)
  - Прямая и обратная польская запись (PN, RPN)
- Поведенческие модели
  - Граф потока управления (control flow graph, CFG)
  - Статическое однократное присваивание (single static assignment, SSA)
  - Граф зависимостей по данным (data dependency graph, DDG)
  - Граф зависимостей программы (program dependency graph, PDG)
  - Трёхадресный код (three-address code, TAC, 3AC)
  - Море узлов (Sea of Nodes)
- Гибридные модели
  - Абстрактный семантический граф (abstract semantic graph, ASG)
  - Граф свойств программы (code property graph, CPG)



Дерево разбора

- Синтаксическое дерево, конкретное синтаксическое дерево, parsing tree
- Является формой представления программы, соответствующей грамматике исходного языка (например, БНФ или РБНФ)
- Строится парсерами написанными вручную или синтезированными генераторами парсеров
- Является наиболее детальным структурным представлением программы
- В исходном виде редко используется для анализа программ из-за громоздкости



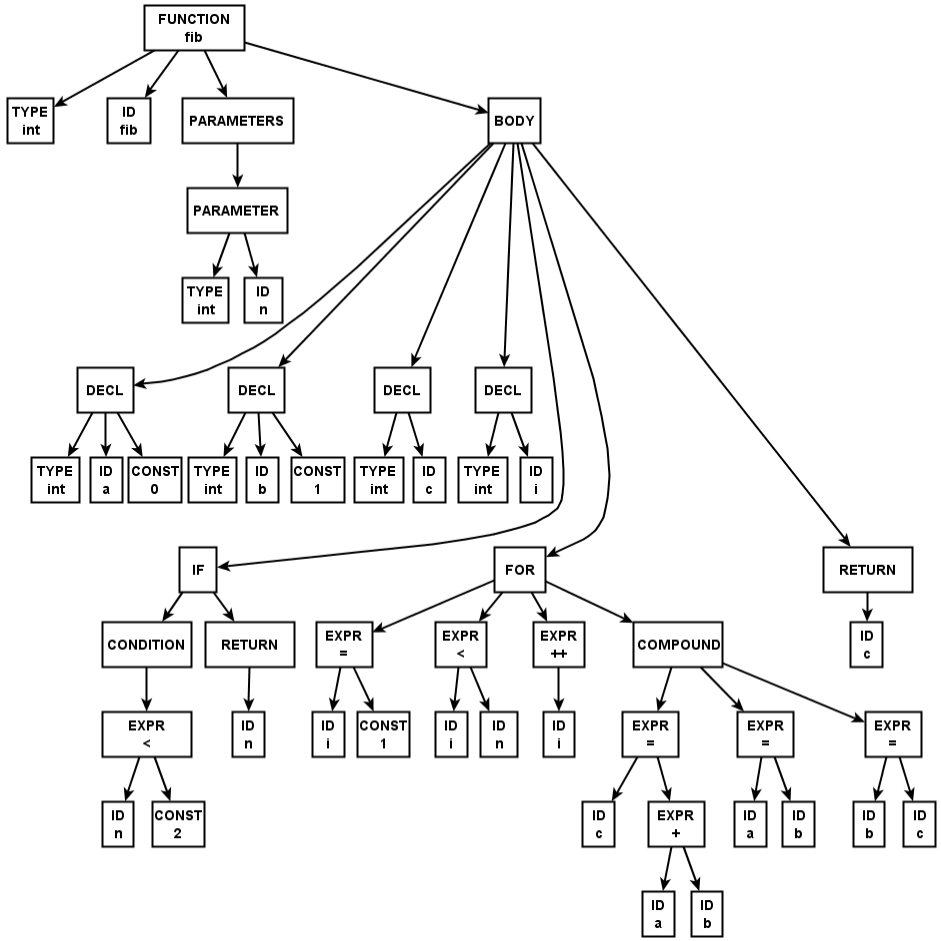
Абстрактное синтаксическое дерево

- Структурное представление программы
- Предложено Джоном Маккарти в 1962 году
- Отличие от синтаксического дерева (дерева разбора):
  - Удаление семантически незначимых элементов
  - Удаление большинства нетерминальных узлов, имеющих одного потомка
  - Замена части терминальных символов атрибутами узлов
  - Модификация группирующих узлов
- По модели АСД легко восстановить исходную программу с точностью до форматирования и комментариев
- Строится из дерева разбора путем постобработки



Абстрактное синтаксическое дерево

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```





Польская запись

- Польская нотация (или польская запись)
- Придумана в 1920х годах польским математиком Я. Лукасевичем
- Используется как альтернативный способ записи арифметических и логических выражений
- Позволяет записывать выражения с функциями с фиксированным числом параметров в бесскобочной форме
- Существует 2 варианта:
  - Прямая польская запись
  - Обратная польская запись





Прямая польская запись

- Polish notation (PN), Normal polish notation (NPN)
- Оператор записывается до операндов

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

№		№	
1	/	10	*
2	+ (–)	11	*
3	neg	12	4
4	B	13	a
5	sqrt	14	c
6	–	15	*
7	*	16	2
8	b	17	a
9	b		



Обратная польская запись

- Reverse Polish notation, RPN
- Оператор записывается после операндов

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

№		№	
1	b	10	*
2	neg	11	–
3	b	12	sqrt
4	b	13	+ (–)
5	*	14	2
6	4	15	a
7	a	16	*
8	c	17	/
9	*		



Польская запись

- Чаще встречается обратная польская запись
- Применяется:
  - в стековых машинах
  - как способ представления синтаксических деревьев



Граф потока управления

- Граф потока управления (ГПУ), Control Flow Graph (CFG)
- Потоки управления программы представляются в виде направленного графа
- Определяется не только структурой программы, но и семантикой операторов
- Два узла графа (**A** и **B**) связаны направленной дугой, если возможно выполнение **B** непосредственно после выполнения **A**
- ГПУ используется для:
  - Генерации кода
  - Оптимизации программ
  - Обнаружения ошибок в программе
  - И т.п.



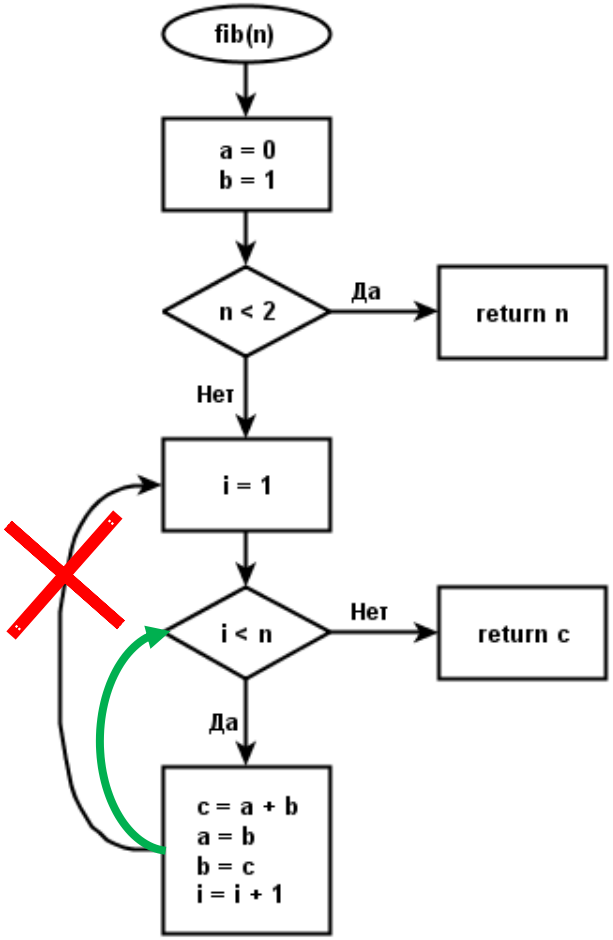
Построение графа потока управления

- ГПУ строится с помощью анализа АСД на основе семантики языковых конструкций
  - Для большинства конструкций строится дуга, соединяющая узлы, соответствующие следующим друг за другом операторам
  - Для части конструкций (ветвления, циклы, вызов функций, переходы и т.п.) – в соответствии с семантикой
- Проблемные конструкции языков программирования:
  - Вычисляемые безусловные переходы (goto)
  - Вызов функции по указателю
  - Полиморфизм
  - Структурная обработка исключений
  - Параллелизм
  - И т.п.



Граф потока управления

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```





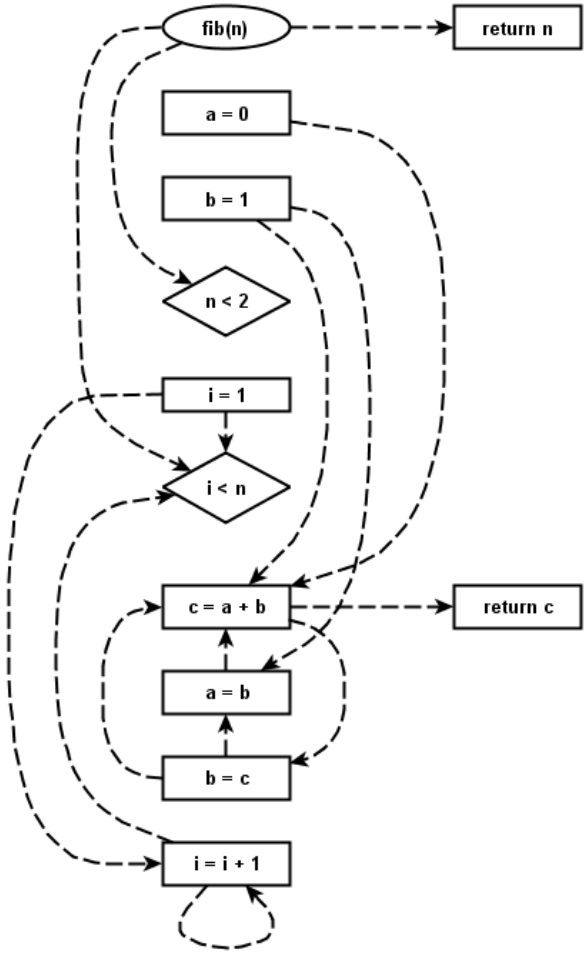
Граф зависимостей по данным

- Граф зависимости по данным (ГЗД), Data Dependency Graph (DDG)
- Отображает связи по данными между объектами программы
- Показывает зависимости между объявлениями, инициализациями, присваиваниями объектов и их использованиями
- Определяется структурой программы, семантикой конструкций
- Используется для:
  - Оптимизации программ
  - Автоматического распараллеливания программ
  - Обнаружения неинициализированных объектов
  - Обнаружения некорректно инициализированных объектов
  - и т.п.



Граф зависимостей по данным

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```



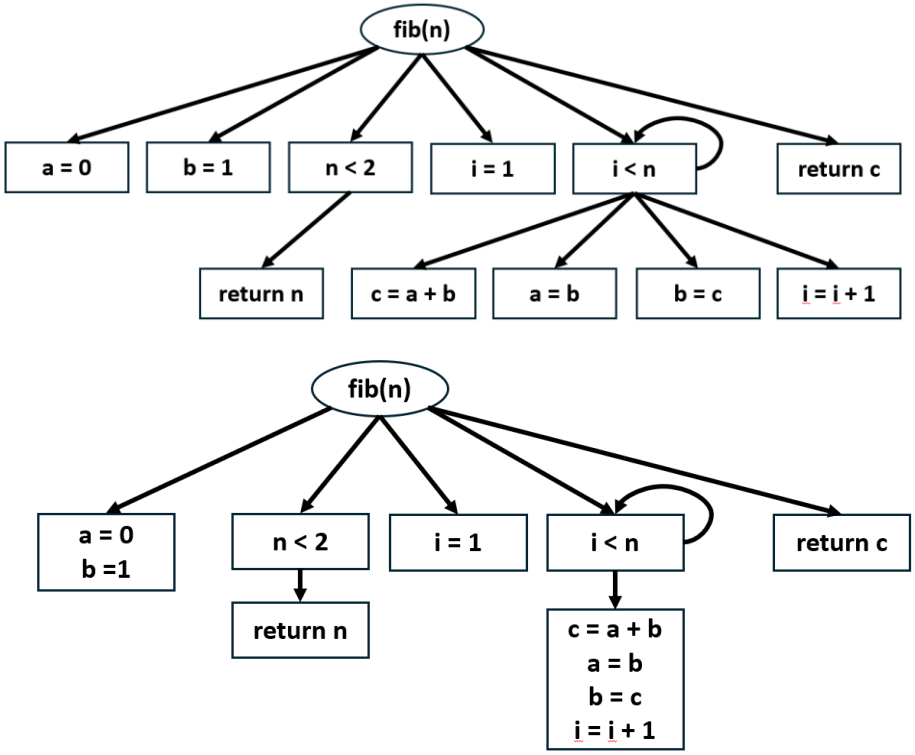


Граф зависимостей по управлению

- Граф зависимостей по управлению (ГЗУ), Control Dependence Graph (CDG)
- Зависимости по управления программы представляются в виде направленного графа
- Два узла графа (**A** и **B**) связаны направленной дугой, если обязательно после выполнения **A** в итоге выполнится **B** (то есть промежуточные условные узлы не повлияют на это)
- A является доминатором над B
- CDG используется для:
  - Оптимизации программ
  - Построения срезов программ (слайсинг)
  - Анализа безопасности
  - И т.п.

Граф зависимостей по управлению

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```



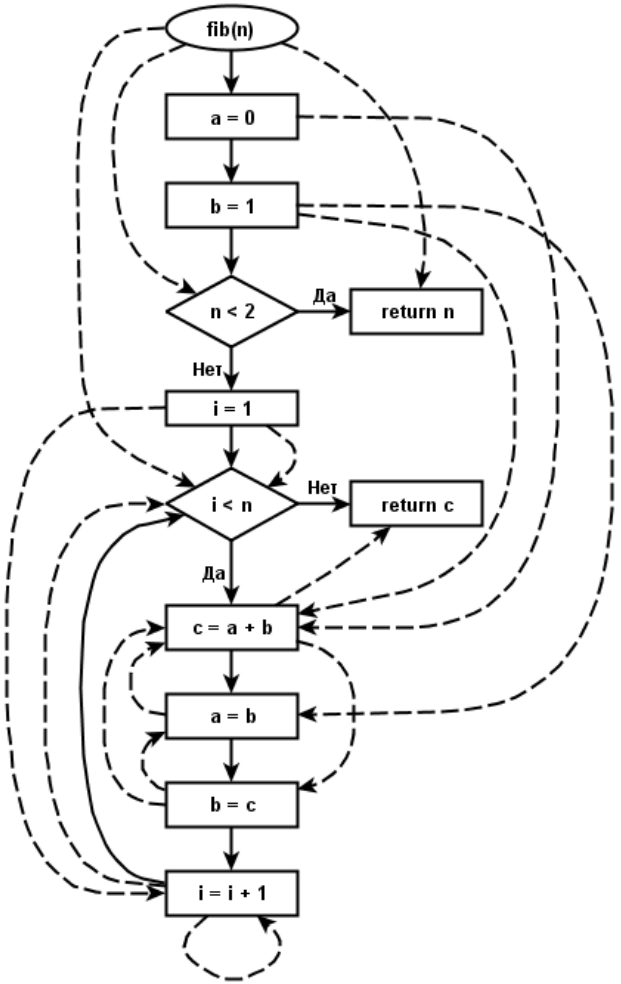


Граф зависимостей программы

- Program Dependence Graph (PDG)
- Является комбинированной моделью
- Существует два варианта графа зависимостей программ
  - CFG +DDG
  - CDG + DDG
- Содержит два типа связей:
  - Зависимости по данным (DDG)
  - Связь по управлению
    - Или связи потока управления (CFG)
    - Или зависимости по управлению (CDG)
- Используется для
  - Оптимизации
  - Генерации кода
  - Анализа программ
  - Верификации
  - Слайсинга
  - И т.п.

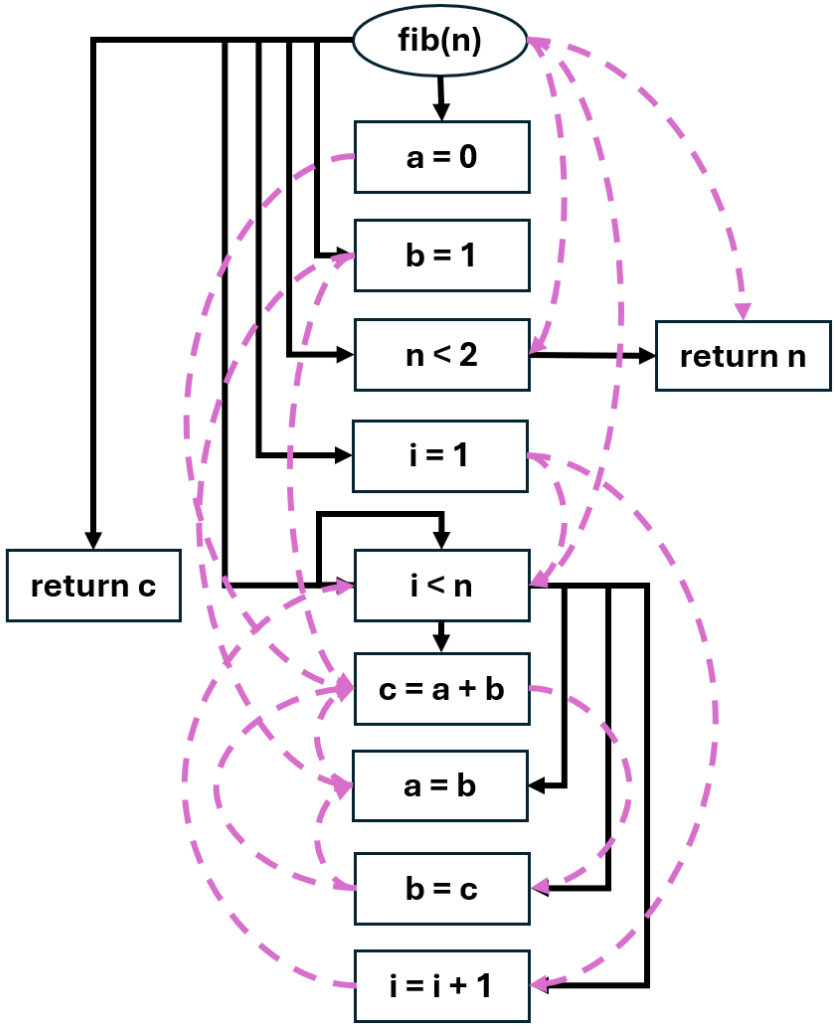
Граф зависимостей программы (1)

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```



Граф зависимостей программы (2)

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```



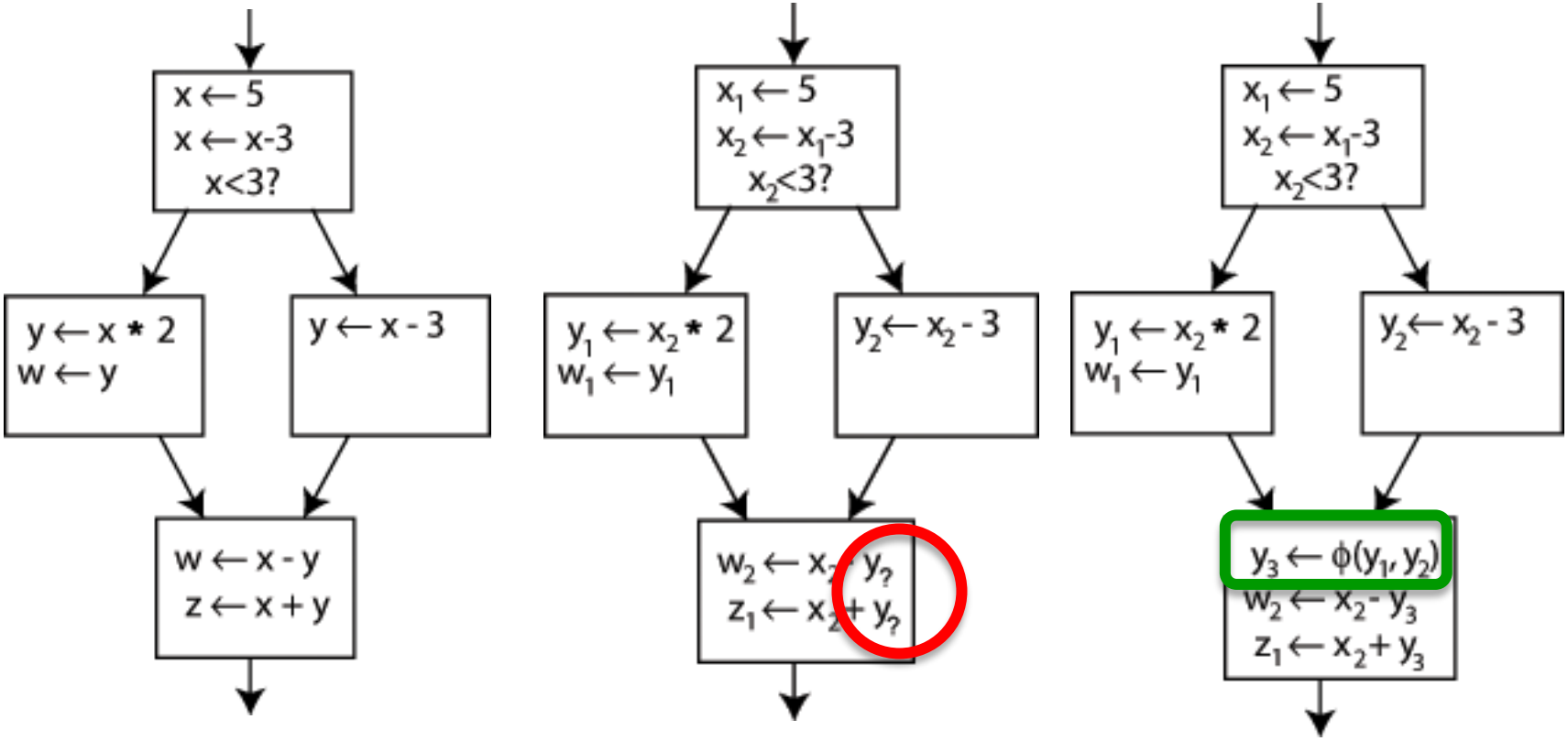


Статическое однократное присваивание

- Статическое однократное присваивание, Static Single Assignment form (SSA)
- Разработано IBM в 1981 году
- Модель, объединяющая граф потока управления и элементы зависимости по данным
- Значения всем объектам программы присваиваются один раз
- Вводится версионирование для каждого очередного присваивания объекта
- При обращении к значению объекта используется последняя версия по ходу графа потока управления
- Неоднозначность разрешается с помощью ϕ-функции



Статическое однократное присваивание





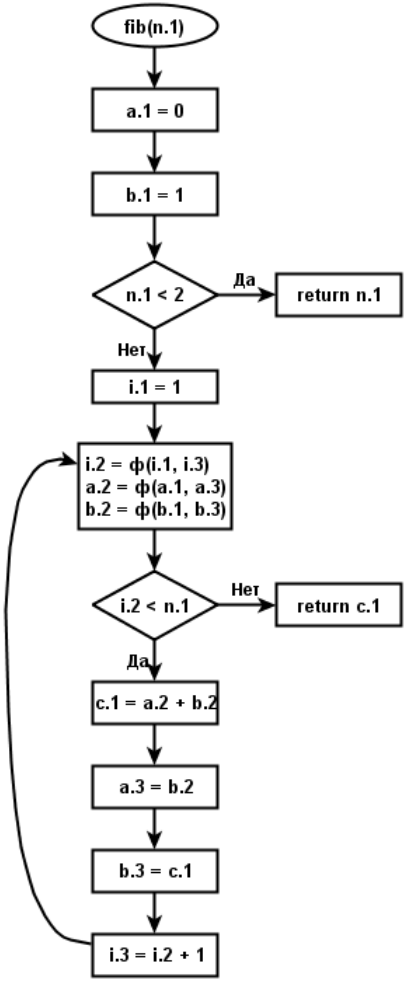
Статическое однократное присваивание

- Используется в компиляторах, статических анализаторах, верификаторах:
  - Распространение констант (constant propagation)
  - Обнаружение удаление мёртвого кода
  - Обнаружение неинициализированных объектов
  - Распараллеливание
  - Генерация кода
  - Преобразование программы / части программы в формулы для последующей верификации
  - И т.п.



Статическое однократное присваивание

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```





Трёхадресный код

- Трёхадресный код, Three-address code, (TAC, 3AC)
- Модель программы, представляющая программу в виде списка конструкций:
  - x := y op z, где
    - y, z – операнды (переменные или константы)
    - op – двуместная операция
    - x – результат (переменная)
- Каждая трёхадресная конструкция:
  - имеет ровно одну операцию
  - второй операнд может отсутствовать
  - результат может отсутствовать



Трёхадресный код

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```

```
    a := 0
    b := 1
    t1 := 2 - n
    ifge t1 goto M3
    i := 0
M1: t2 := i - n
    ifge t2 goto M2
    c := a + b
    a := b
    b := c
    i := i + 1
    goto m1
M2: result := c
    return
M3: result := n
    return
```



Трёхадресный код

- Операции в трёхадресном коде
  - двуместные арифметические, логические операции
  - одноместные арифметические, логические операции
  - индексация, получение адреса, разадресация
  - присваивание
  - безусловный переход
  - условные переходы
  - вызов процедур
- Представления трёхадресного кода
  - в виде тетрад
  - в виде триад
  - в виде косвенных триад



Трёхадресный код. Представление в виде тетрад

№	Op	Arg1	Arg2	Result
0	:=	0		a
1	:=	1		b
2	-	2	n	t1
3	ifge	t1	(14)	
4	:=	0		i
5	-	i	n	t2
6	ifge	t2	(12)	
7	+	a	b	c
8	:=	b		a
9	:=	c		b
10	+	i	1	i
11	goto	(5)		
12	:=	c		result
13	return			
14	:=	n		result
15	return			



Трёхадресный код. Представление в виде триад

	Op	Arg1	Arg2
0	load	0	
1	load	1	
2	–	2	n
3	ifge	(2)	(15)
4	load	0	
5	–	(4)	n
6	ifge	(5)	(13)
7	+	(0)	(1)
8	:=	(0)	(1)
9	:=	(1)	(7)
10	+	(4)	1
11	:=	(4)	(10)
12	goto	(5)	
13	:=	result	(7)
14	return		
15	:=	result	n
16	return		

a

b

t_1

i

t_2

c

t_3

Относительные адреса. Могут разъезжаться при оптимизациях



Трёхадресный код. Представление в виде косвенных триад

	Op	Arg1	Arg2
0	load	0	
1	load	1	
2	–	2	n
3	ifge	(2)	(7)
4	load	0	
5	–	(3)	n
6	ifge	(4)	(8)
7	+	(0)	(1)
8	:=	(0)	(1)
9	:=	(1)	(5)
10	+	(3)	1
11	:=	(3)	(6)
12	goto	(9)	
13	:=	result	(5)
14	return		
15	:=	result	n
16	return		

Var	Line
0	0
1	1
2	2
3	4
4	5
5	7
6	10
7	15
8	13
9	5



Использование трёхадресного кода

- при генерации двоичного кода
- при генерации байт-кода



Море узлов

- Море узлов – Sea of Nodes
- Модель предложена Клиффордом Кликом в 1995 году
- Основная идея
  - Всё – узлы
  - Нет **явных** связей по управлению
- Константы, переменные, операции, функции условия задаются узлами графа
- Базовые блоки не используются
- Есть специальный узел, объединяющий потоки (как фи-функция в SSA)



Море узлов

- Ребра отображают все виды семантических связей:
  - зависимости по данным
  - зависимости по управлению
  - определение-использование
  - переменная-тип
  - и т.п.
- Финальный порядок инструкций при кодогенерации определяется на этапе глобального движения кода (global code motion)
- Используется
  - при компиляции
  - при оптимизациях

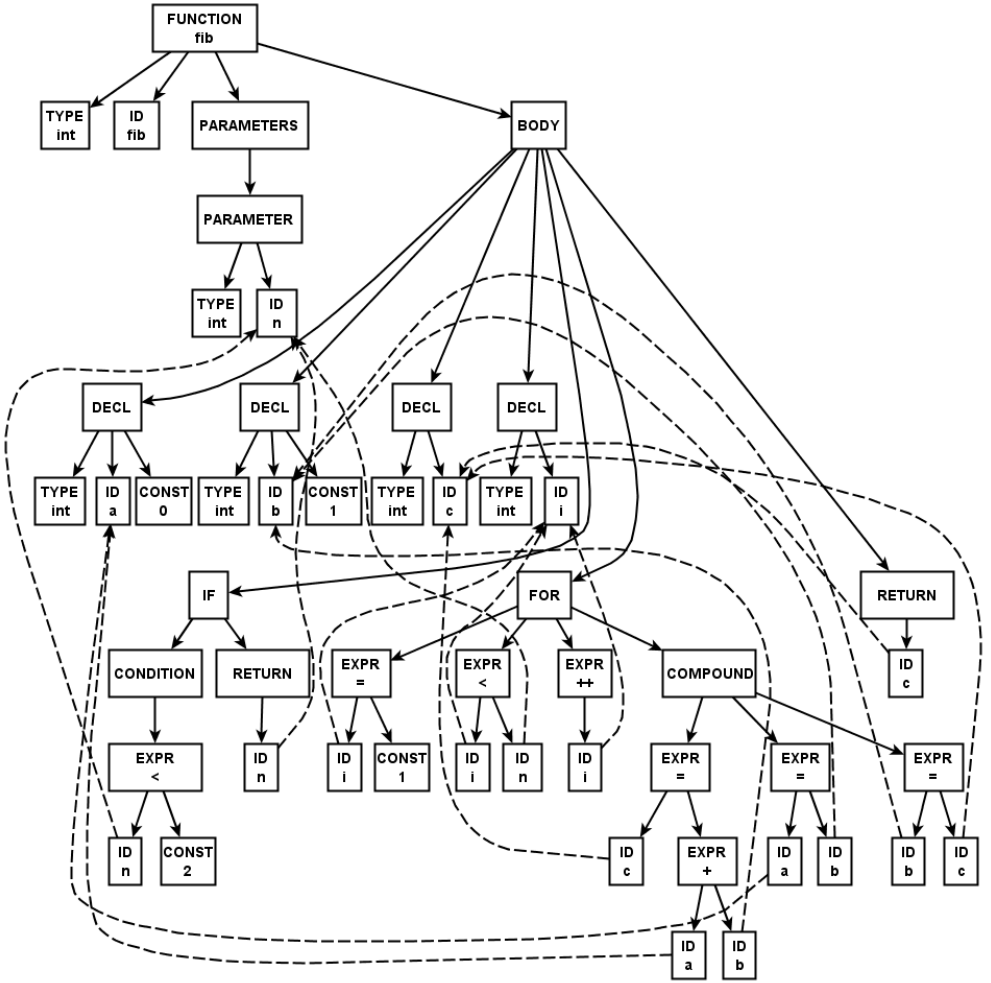


Абстрактный семантический граф

- Abstract Semantic Graph
- Наиболее общая модель, объединяющая структурные и поведенческие компоненты
- Абстрактный синтаксический граф обогащается семантической информацией:
  - Связи по данным
  - Связи по управлению
  - Типами данных
  - И т.п.
- В общем случае, строгого определения ASG нет

Абстрактный семантический граф

```
int fib(int n)
{
  int a = 0, b = 1, c, i;
  if (n < 2) return n;
  for (i = 1; i < n; i++)
  {
    c = a + b;
    a = b;
    b = c;
  }
  return c;
}
```





Граф свойств программы

- Code Property Graph, CPG
- Является четко специфицированной реализацией ASG
- Предложен в 2013 году в Гёттингенском университете
- Основные идеи:
  - Объединить структурное представление, управляющие связи и связи по данным в одной модели
  - Унифицировать и специфицировать структуру графа
  - Предоставить унифицированный интерфейс доступа к свойствам модели
- Оригинальный CPG является частью проекта Joern и формально описан здесь: https://cpg.joern.io/



Граф свойств программы

- Метаданные (MetaData)
  - Задаются узлом (один на программу)
  - Содержат информацию о проекте: хэш, корневая директория и т.п.
- Файловая система (FileSystem)
  - Задается узлами и дугами
  - Информация о файлах проекта
  - Информация о связях узлов CPG с файлами (в том числе и локации)
- Конфигурации (Configuration)
  - Задаются узлами
  - Описывают специфические конфигурационные файлы, требуемые специфическим анализам
- Пространства имен (Namespace)
  - Задаются узлами
  - Представляют пространства имен программы



Граф свойств программы

- Метод (Method)
  - Задается узлами
  - Описывает сигнатуру метода (функции), в том числе параметры и возвращаемое значение
- Тип (Type)
  - Задается узлами и дугами
  - Описывает систему типов
  - Содержит информацию о полях, типовых параметрах, параметризации, наследовании и т.п.
- АСД (Ast)
  - Задается узлами и дугами
  - Описывает все узлы АСД (декларации, управление, вызовы и т.п.)
- Граф вызовов (CallGraph)
  - Задается дугами
  - Описывает отношения вызова между методами



Граф свойств программы

- ГПУ (CFG)
  - Задается узлами и дугами
  - Описывает связи по управлению
- Доминаторы (Dominator)
  - Задаются дугами
  - Содержат вспомогательную информацию для построения семантических связей
- Зависимости программы (PDG)
  - Задается дугами
  - Описывают зависимости по данным и по управлению
- Выводы (Findings)
  - Задаются узлами
  - Содержат результаты анализов (баги и уязвимости)
- Ярлыки (Shotrcuts)
  - Задаются дугами
  - Содержат короткие связи между узлами (дуга к методу, типу и т.п.)



Граф свойств программы

- Тэги и местоположения (TagsAndLocation)
  - Задаются узлами и дугами
  - Описывают произвольные промежуточные результаты анализов
- Привязки (Binding)
  - Задаются узлами дугами
  - Используются при разрешении типов
- Аннотации (Annotation)
  - Задаются узлами
  - Описывают аннотации языка
- Декларации
  - Задаются узлами и дугами
  - Описывают связи между идентификаторами и объектами в программе



Другие модели

- Конкретные анализаторы и верификаторы могут использовать другие модели:
  - Модель Крипке
  - Автоматы Бюхи
  - Модели на основе Datalog
  - и т.п.



Правила выбора модели

- Должна содержать всю необходимую для решаемой задачи семантическую информацию
- Не должна содержать лишнюю информацию
- Должна обеспечивать быстрый поиск информации
  - Это особенно важно для больших программных проектов